

強化及保護雲端到雲端整合 | Cloud-to-cloud | Google Home Developers

來源：<https://developers.home.google.com/codelabs/smarthome-traits?hl=zh-tw>

步驟數：8

瞭解如何透過可自訂的裝置特徵強化雲端到雲端整合功能並加以保護，並使用雙重驗證來確保安全。

目錄

1. [1. 事前準備](#)
2. [2. 開始使用](#)
3. [3. 執行範例應用程式](#)
4. [4. 新增模式](#)
5. [5. 新增切換按鈕](#)
6. [6. 回報錯誤和例外狀況](#)
7. [7. 新增雙重使用者驗證](#)
8. [8. 恭喜](#)

步驟 1 · 約 0 分鐘

強化及保護雲端對雲端整合

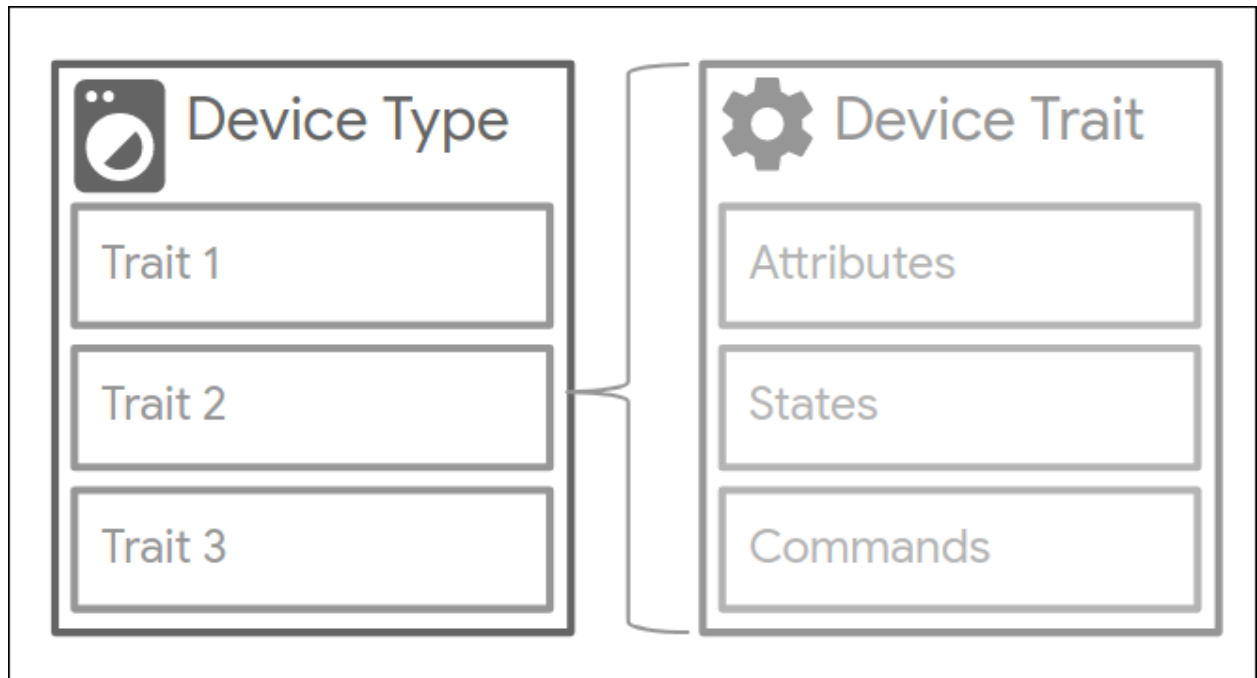
程式碼研究室簡介

subject上次更新時間：5月 12, 2026

account_circle作者：Google 員工

1. 事前準備

雲端整合功能會使用[裝置類型](#)，讓 Google 助理瞭解裝置應使用的文法。[裝置特徵](#)會定義裝置類型的功能。裝置會繼承新增至整合的每個裝置特徵狀態。



您可以將任何支援的特徵連結至所選裝置類型，自訂使用者裝置的功能。如要在動作中實作裝置架構目前未提供的自訂特徵，可以使用「模式」和「切換」特徵，透過您定義的自訂名稱控制特定設定。

除了型別和特徵提供的基本控制功能外，智慧型住宅 API 還提供額外功能，可提升使用者體驗。意圖失敗時，錯誤回應會提供詳細的使用者意見回饋。雙重使用者驗證會擴充這些回應，並為所選裝置特徵新增額外安全措施。只要將特定錯誤回應傳送至 Google 助理發出的驗證碼區塊，Cloud-to-Cloud 整合服務就能要求額外授權，才能完成指令。

必要條件

- [建立雲端對雲端整合開發人員指南](#)
- [智慧型住宅洗衣機](#)程式碼研究室
- [裝置類型和特徵](#)開發人員指南

建構項目

在本程式碼研究室中，您將使用 Firebase 部署預先建構的智慧型住宅整合功能，然後瞭解如何為[智慧型住宅洗衣機](#)新增非標準特徵，以便設定洗衣量和快速模式。您也會實作錯誤和例外狀況回報功能，並瞭解如何透過雙重使用者驗證，強制執行口頭確認來開啟洗衣機。

課程內容

- 如何在整合中加入模式和切換開關特徵

- [如何回報錯誤和例外狀況](#)
- [如何啟用雙重使用者驗證](#)

軟硬體需求

- 網路瀏覽器，例如 [Google Chrome](#)
- 已安裝 [Google Home 應用程式](#) 的 iOS 或 Android 裝置
- [Node.js](#) 24 以上版本
- [Google 帳戶](#)
- Google Cloud [帳單帳戶](#)
- [Firebase](#) 的 Blaze (即付即用) 方案。

注意：GitHub 存放區中有一些依附元件未列於此處。您必須遵循 GitHub 存放區 `package.json` 檔案中指定的依附元件版本。

步驟 2 · 約 5 分鐘

2. 開始使用

啟用活動控制項

如要使用 Google 助理，您必須與 Google 分享特定活動記錄。Google 助理需要這項資料才能正常運作，但分享資料的要求並非 SDK 專屬。如要分享這項資料，請建立 [Google 帳戶](#) (如果還沒有帳戶)。您可以使用任何 Google 帳戶，不一定是開發人員帳戶。

開啟要與 Google 助理搭配使用的 Google 帳戶的[活動控制項頁面](#)。

確認已啟用下列切換開關：

- **網路和應用程式活動** - 此外，請務必勾選「包括 Chrome 歷史記錄，以及採用 Google 服務的網站、應用程式和裝置上的活動記錄」核取方塊。
- **裝置資訊**
- **語音和音訊活動**

您必須在手機和開發人員控制台使用相同的 Google 帳戶，才能完整執行這個專案。

建立雲端對雲端整合專案

1. 前往[開發人員控制台](#)。
2. 按一下「建立專案」，輸入專案名稱，然後按一下「建立專案」。

Create a project

Create a project to get started.

Project name

30 characters max

Create project

Already have a Works with Assistant, cloud, or Local Home SDK project? [Import existing project](#).

選取雲端對雲端整合

在開發人員控制台的「專案首頁」中，選取「Cloud-to-cloud」下方的「新增雲端對雲端整合」。

Cloud-to-cloud

Cloud-connected devices that support smart home Actions. Devices can be controlled through the Google Home app and Google Assistant. [Learn more](#)

[+ Add cloud-to-cloud integration](#)

No cloud-to-cloud integrations yet.

這個專案使用 [Cloud Functions for Firebase](#)，因此您必須將帳單帳戶與專案建立關聯。Actions 專案預設不會建立帳單帳戶。詳情請參閱「[建立新的帳單帳戶](#)」一文。

安裝 Firebase CLI

Firebase 指令列介面 (CLI) 可讓您在本地提供網頁應用程式，並將網頁應用程式部署至 Firebase 託管。

如要安裝 CLI，您必須先安裝 [npm](#)，這通常會隨附於 [Node.js](#)。

如要安裝 CLI，請從終端機執行下列 npm 指令：

```
npm install -g firebase-tools
```

如果無法正常運作，建議您變更 npm 權限。

如要確認 CLI 是否已正確安裝，請執行下列指令：

```
firebase --version
```

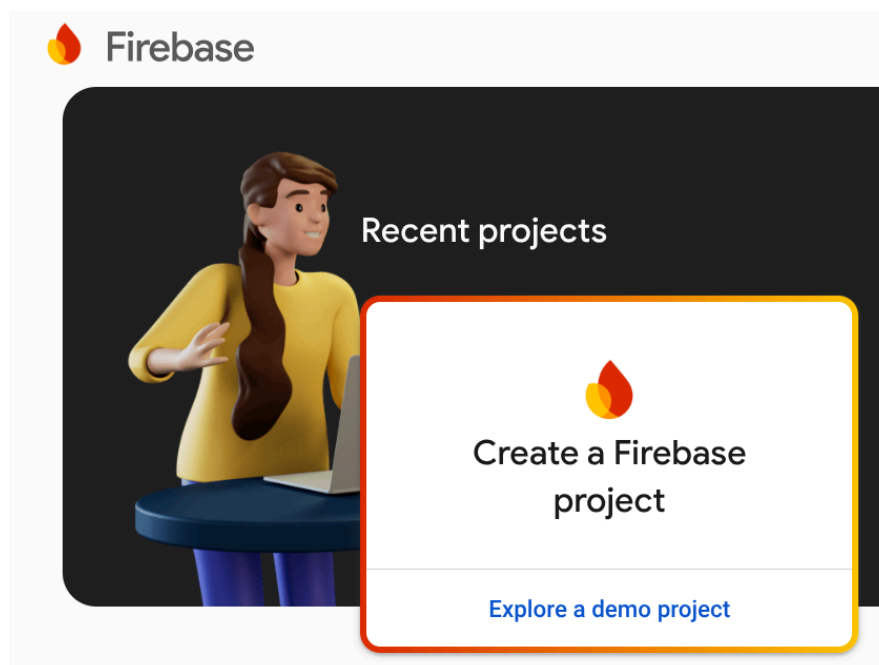
執行下列指令，透過 Google 帳戶授權 Firebase CLI：

```
firebase login
```

將 Firebase 新增至 Google Home 開發人員控制台專案

方法 1：透過 Firebase 控制台

1. 前往 [Firebase](#)。
2. 點選「建立 Firebase 專案」。



3. 在「建立專案」畫面中，按一下「將 Firebase 新增到 Google Cloud 專案」。

× Create a project

Let's start with a name for your project[?]

Enter your project name

my-awesome-project-id

Already have a Google Cloud project?
[Add Firebase to Google Cloud project](#)

Continue

4. 在「開始使用」畫面中，選取您剛在 Google Home 開發人員控制台中建立的 Google Cloud 專案，然後按一下「繼續」。

× Get started

Let's start with choosing your Google Cloud project

Select a Google Cloud project

Not what you wanted?
[Create a new project instead](#)

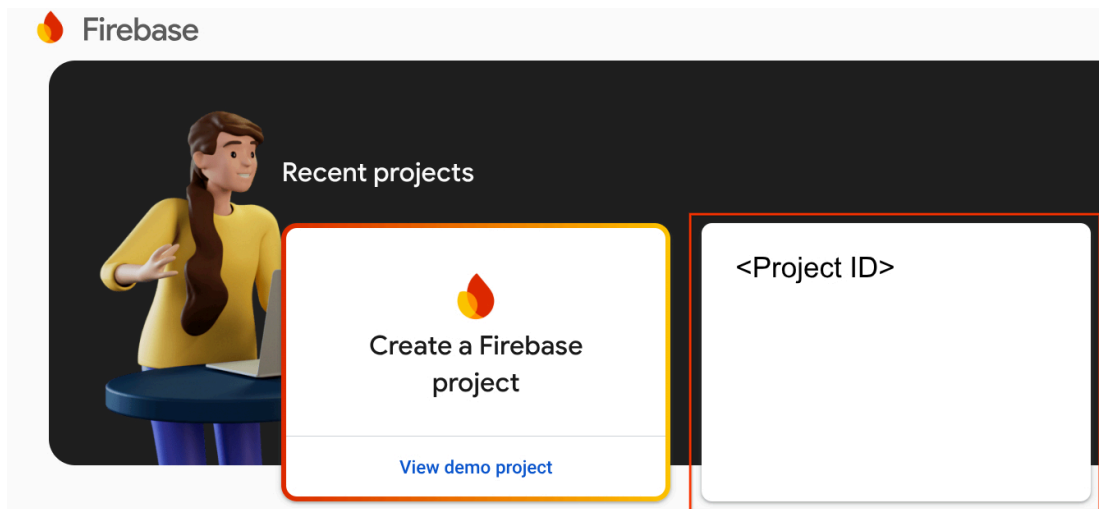
Continue

方法 2：透過 Firebase CLI

```
firebase projects:addfirebase
```

選取您剛建立的 Google Home 開發人員控制台專案，然後新增 Firebase。

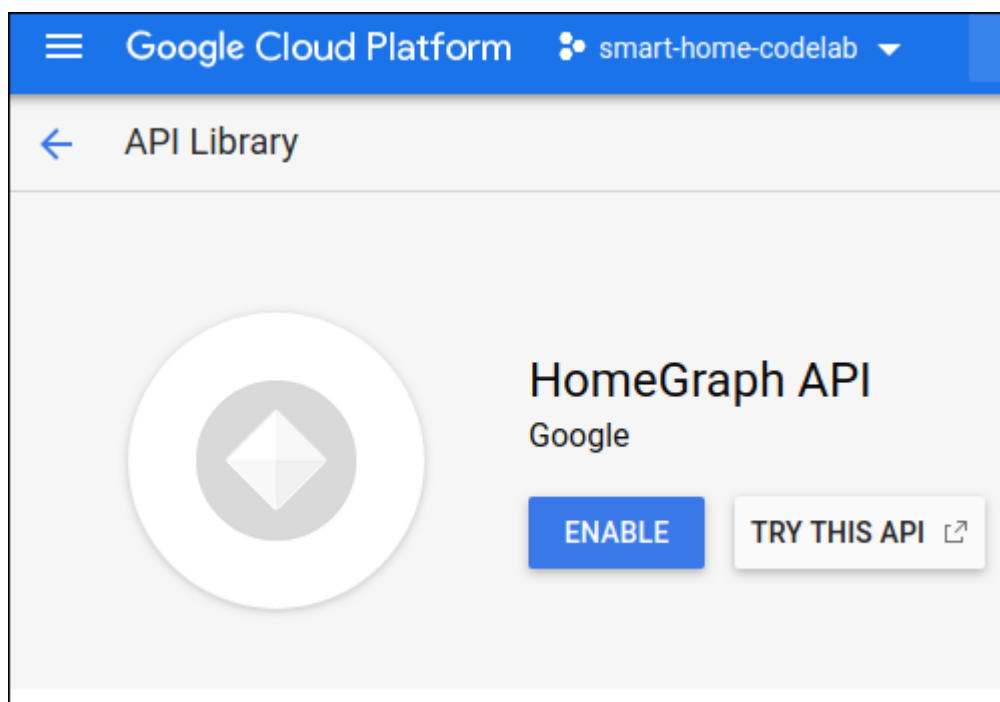
將 Firebase 新增至 Google Home 開發人員控制台專案後，Firebase 控制台就會顯示該專案。Firebase 專案的專案 ID 會與 Google Home 開發人員控制台專案 ID 一致。



啟用 HomeGraph API

[HomeGraph API](#) 可讓您在使用者 Home Graph 中儲存及查詢裝置和裝置狀態。如要使用這項 API，請先開啟 Google Cloud 控制台並[啟用 HomeGraph API](#)。

在 Google Cloud 控制台中，請務必選取與動作相符的專案 `<firebase-project-id>`。接著，在 HomeGraph API 的「API 程式庫」畫面中，按一下「啟用」。



3. 執行範例應用程式

現在您已設定開發環境，可以部署入門專案，確認一切設定正確無誤。

取得原始碼

點選下方連結，即可在開發機器上為本程式碼研究室下載範例：

...或者，您也可以從指令列複製 GitHub 存放區：

```
git clone https://github.com/google-home/smarthome-traits.git
```

將下載的 ZIP 檔案解壓縮。

注意：程式碼研究室存放區包含兩個目錄：

- `washer-start`：您要建構的起始程式碼。
- `washer-done` — 完成的自訂洗衣機應用程式完整程式碼。

如要執行完成的應用程式，您仍需在[開發人員控制台](#)中建立專案。

關於專案

入門專案包含下列子目錄：

- `public`：前端 UI，可輕鬆控制及監控智慧洗衣機的狀態。
- `functions`：透過 Cloud Functions for Firebase 和 Firebase 即時資料庫，管理智慧型洗衣機的雲端服務已全面實作。

提供的雲端履行服務包含 `index.js` 中的下列函式：

- `fakeauth`：帳戶連結的授權端點
- `faketoken`：用於帳戶連結的權杖端點
- `smarthome`：智慧型住宅意圖履行端點
- `reportstate`：在裝置狀態變更時叫用 Home Graph API
- `requestsync`：使用者裝置更新時，不必重新連結帳戶

提示：尋找 `TODO` 標記。

範例專案會顯示提示，指出需要變更或新增程式碼的位置。例如：

```
// TODO: Implement custom trait
```

連結至 Firebase

前往 `washer-start` 目錄，然後使用整合專案設定 Firebase CLI：

```
cd washer-start
firebase use <project-id>
```

提示：如果您不知道雲端對雲端整合的專案 ID，可以在 Firebase 控制台的「專案總覽」>「專案設定」>「一般」分頁標籤中找到。

設定 Firebase 專案

初始化 Firebase 專案。

```
firebase init
```

選取 CLI 功能、「即時資料庫」和「函式」功能。

```
? Which Firebase features do you want to set up for this directory? Press Space to
select features, then Enter
to confirm your choices. (Press <space> to select, <a> to toggle all, <i> to invert
selection, and <enter> to
proceed)
>( ) Data Connect: Set up a Firebase Data Connect service
( ) Firestore: Configure security rules and indexes files for Firestore
( ) Genkit: Setup a new Genkit project with Firebase
(*) Functions: Configure a Cloud Functions directory and its files
( ) App Hosting: Configure an apphosting.yaml file for App Hosting
( ) Hosting: Configure files for Firebase Hosting and (optionally) set up GitHub
Action deploys
( ) Storage: Configure a security rules file for Cloud Storage
( ) Emulators: Set up local emulators for Firebase products
( ) Remote Config: Configure a template file for Remote Config
( ) Extensions: Set up an empty Extensions manifest
(*) Realtime Database: Configure a security rules file for Realtime Database and
(optionally) provision
default instance
( ) Data Connect: Set up a Firebase Data Connect service
( ) Firestore: Configure security rules and indexes files for Firestore
```

這會為專案初始化必要的 API 和功能。

系統提示時，請初始化即時資料庫。您可以沿用資料庫執行個體的預設位置。

? It seems like you haven't initialized Realtime Database in your project yet. Do you want to set it up?

Yes

? Please choose the location for your default Realtime Database instance:

us-central1

由於您使用的是入門專案程式碼，請選擇安全性規則的預設檔案，並確保不會覆寫現有的資料庫規則檔案。

? File database.rules.json already exists. Do you want to overwrite it with the Realtime Database Security Rules for <project-ID>-default-rtdb from the Firebase Console?

No

如果重新初始化專案，系統詢問是否要初始化或覆寫程式碼集時，請選取「覆寫」。

? Would you like to initialize a new codebase, or overwrite an existing one?

Overwrite

設定函式時，請使用預設檔案，並確保不會覆寫專案範例中現有的 **index.js** 和 **package.json** 檔案。

? What language would you like to use to write Cloud Functions?

JavaScript

? Do you want to use ESLint to catch probable bugs and enforce style?

No

? File functions/package.json already exists. Overwrite?

No

? File functions/index.js already exists. Overwrite?

No

如果重新初始化專案，系統詢問是否要初始化或覆寫 functions/.gitignore 時，請選取「No」。

? File functions/.gitignore already exists. Overwrite?

No

? Do you want to install dependencies with npm now?

Yes

如果不小心啟用了 ESLint，可以透過以下兩種方法停用：

1. 使用 GUI 前往專案底下的 `../functions` 資料夾，選取隱藏檔案 `.eslintrc.js` 並刪除。請勿與名稱相似的 `.eslintrc.json` 混淆。

2. 使用指令列：

```
cd functions  
rm .eslintrc.js
```

部署至 Firebase

安裝依附元件並設定專案後，即可首次執行應用程式。

```
firebase deploy
```

您應該會看到下列控制台輸出內容：

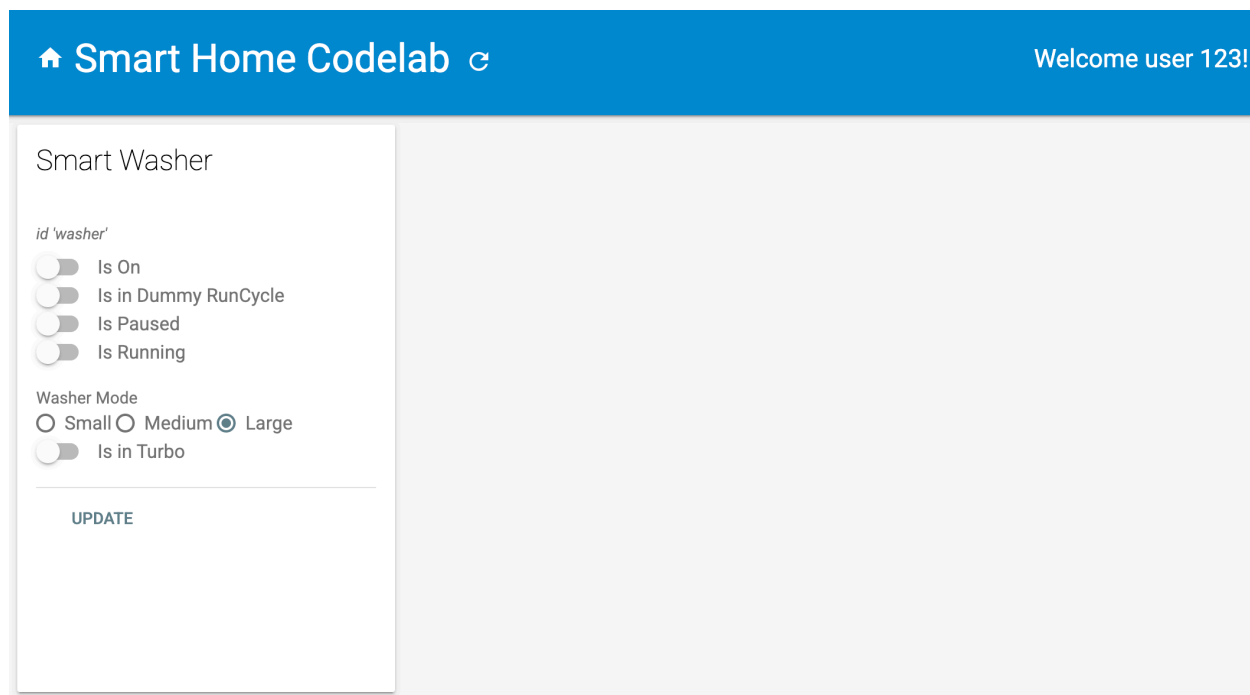
```
...  
  
✓ Deploy complete!  
  
Project Console: https://console.firebase.google.com/project/<project-id>/overview  
Hosting URL: https://<project-id>.web.app
```

這個指令會部署網頁應用程式，以及多個 [Cloud Functions for Firebase](#)。

注意：如果尚未為專案設定帳單，您必須升級至 **Blaze (即付即用)** 方案，[Firebase 函式](#)才能完成專案部署。

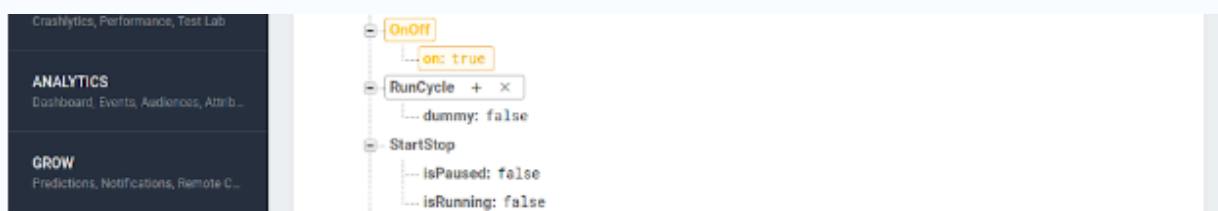
注意：專案設定可能需要幾分鐘才能完全傳播。如果嘗試部署時看到「HTTP Error: 403」，請稍候一分鐘再試一次。

在瀏覽器中開啟「Hosting URL」(主機網址) (<https://<firebase-project-id>.web.app>)，即可查看網頁應用程式。您會看到下列介面：



這個網頁版 UI 代表第三方平台，可查看或修改裝置狀態。如要開始將裝置資訊填入資料庫，請按一下「更新」。頁面上不會顯示任何變更，但洗衣機的目前狀態會儲存在資料庫中。

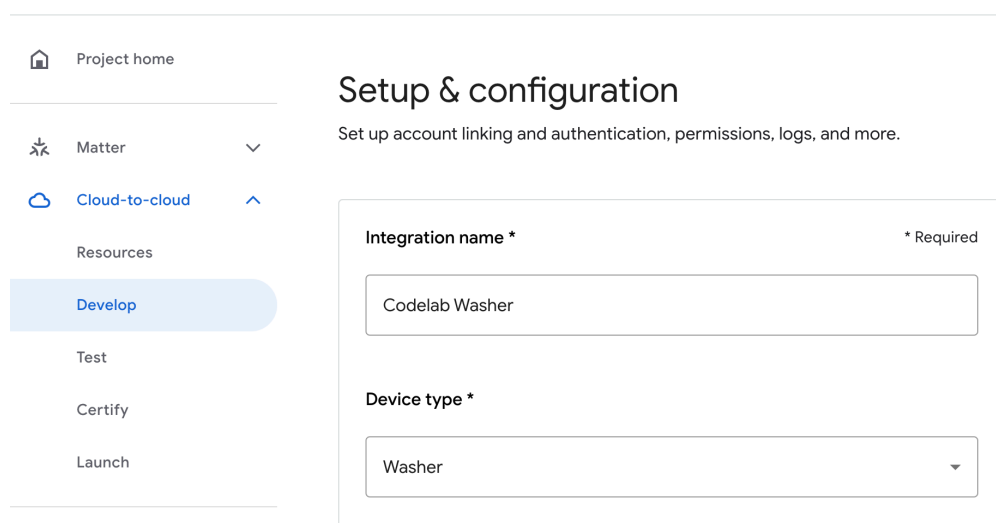
注意：虛擬裝置狀態會使用 [Firebase 即時資料庫](#) 儲存。您可以使用 [Firebase 控制台](#) 直接查看資料庫內容。在左側工具列中，依序點選「Build」>「即時資料庫」。洗衣機狀態的任何更新都會顯示在這裡。



現在請使用 [開發人員控制台](#)，將部署的雲端服務連結至 Google 助理。

設定開發人員控制台專案

在「開發」分頁中，為互動新增「顯示名稱」。這個名稱會顯示在 Google Home 應用程式中。



在「應用程式品牌宣傳」下方，上傳應用程式圖示的 `png` 檔案，大小為 144 x 144 像素，並命名為 `.png`。

App branding

Your service name (the integration name above) and app icon will appear in the Google Home app.

App icon *

File name must be the project ID (e.g., projectid.png). [View icon requirements](#)



File type: PNG transparent

File size: 144 x 144px

Upload image

如要啟用「帳戶連結」，請使用下列帳戶連結設定：

用戶端 ID	ABC123
用戶端密鑰	DEF456
授權網址	<code>https://us-central1-<project-id>.cloudfunctions.net/fakeauth</code>
權杖網址	<code>https://us-central1-<project-id>.cloudfunctions.net/faketoken</code>

請務必將網址更新為您的專案 ID。

Account Linking

Authentication allows you to link your users' Google Accounts with their accounts in your authentication system. Then you can identify your users when your fulfillment receives a smart home intent.

 [Start here.](#) Read the OAuth 2.0 account linking guide before continuing.

OAuth Client ID *

String that identifies the request origin as Google.

Client ID*
ABC123

Client secret *

Secret string that you registered with Google for your Client ID.

Client secret*
.....

Authorization URL *

Endpoint for your sign-in web page that supports OAuth 2.0 code or implicit flows.

Authorization URL*
`https://us-central1-<project-id>.cloudfunctions.net/fakeauth`

Please enter a valid https link

Token URL *

OAuth 2.0 endpoint for token exchange.

Token URL*
`https://us-central1-<project-id>.cloudfunctions.net/faketoken`

Please enter a valid https link

在「Cloud 實現網址」下方，輸入提供智慧住宅意圖實現功能的 Cloud 函式網址。

`https://us-central1-<project-id>.cloudfunctions.net/smarthome`

Cloud fulfillment URL

Enter the fulfillment URL used to process your smart home intents.

Fulfillment URL *

`https://us-central1-<project-id>.cloudfunctions.net/smarthome`

Please enter a valid https link

按一下「儲存」儲存專案設定，然後按一下「下一步：測試」，在專案中啟用測試。

Developer Console

smarthome-washer-dc

Project home

Matter

Cloud-to-cloud

Resources

Develop

Test

Certify

Launch

Analytics

Develop

Edit the integration.

Name	Status	Last modified ↑	
Codelab Washer	Draft	5:43 PM - Oct 2, 2024	Edit

[Next: Test](#)

Related content

Get started

Learn how to properly set up your environment to ensure your implementation will be a success.

[Learn more](#)

Supported devices

See which devices and traits are supported across apps and services.

[Learn more](#)

Codelabs

Get hands-on, step-by-step guidance with our technical product tutorials.

[Go to codelab](#)

現在可以開始實作必要的 Webhook，將裝置狀態與 Google 助理連結。

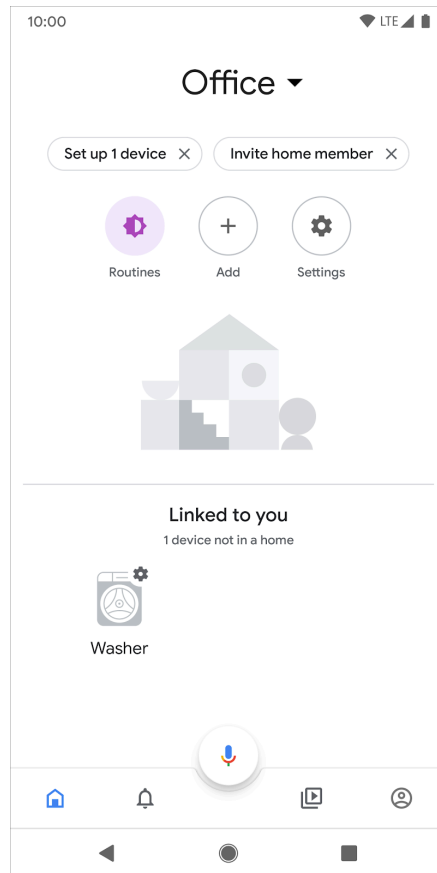
連結至 Google 助理

如要測試雲端對雲端整合，您必須將專案連結至 Google 帳戶。這樣一來，您就能透過登入相同帳戶的 Google 助理介面和 Google Home 應用程式進行測試。

重要事項：本程式碼研究室包含帳戶連結的實作項目，但不會實際檢查使用者憑證。在實際工作環境中，您應實作 OAuth 2.0 通訊協定，確保裝置安全無虞。

1. 在手機上開啟 Google 助理設定。請注意，您登入的帳戶應與控制台的帳戶相同。
2. 依序前往「Google 助理」>「設定」>「居家控制」（位於「Google 助理」下方）。
3. 按一下右上方的搜尋圖示。
4. 使用 **[test]** 前置字元搜尋測試應用程式，找出特定測試應用程式。
5. 選取該項目。Google 助理接著會向服務進行驗證，並傳送 **sync** 要求，請服務提供使用者的裝置清單。

開啟 Google Home 應用程式，確認是否能看到洗衣機裝置。



注意：如果在連結步驟中遇到問題，可以使用 [Cloud Logging](#) 查看錯誤記錄。

確認你可以在 Google Home 應用程式中使用語音指令控制洗衣機。此外，雲端服務的網頁前端 UI 應會顯示裝置狀態變更。

現在你已部署基本洗衣機，可以自訂裝置上可用的模式。

步驟 4 · 約 10 分鐘

4. 新增模式

`action.devices.traits.Modes` 特徵可讓裝置擁有任意數量的模式設定，但一次只能設定一個。你將在洗衣機中新增模式，定義洗衣量大小：小、中或大。

更新 SYNC 回應

您需要在 `functions/index.js` 的 SYNC 回應中新增有關新特徵的資訊。這項資料會顯示在 `traits` 陣列和 `attributes` 物件中，如下列程式碼片段所示。

`index.js`

```

app.onSync(body => {
  return {
    requestId: 'ff36a3cc-ec34-11e6-b1a0-64510650abcf',
    payload: {
      agentUserId: USER_ID,
      devices: [{
        id: 'washer',
        type: 'action.devices.types.WASHER',
        traits: [
          'action.devices.traits.OnOff',
          'action.devices.traits.StartStop',
          'action.devices.traits.RunCycle',
          // Add Modes trait
          'action.devices.traits.Modes',
        ],
        name: { ... },
        deviceInfo: { ... },
        attributes: {
          pausable: true,
          //Add availableModes
          availableModes: [{
            name: 'load',
            name_values: [{
              name_synonym: ['load'],
              lang: 'en',
            }],
            settings: [{
              setting_name: 'small',
              setting_values: [{
                setting_synonym: ['small'],
                lang: 'en',
              }]
            }, {
              setting_name: 'medium',
              setting_values: [{
                setting_synonym: ['medium'],
                lang: 'en',
              }]
            }, {
              setting_name: 'large',
              setting_values: [{
                setting_synonym: ['large'],
                lang: 'en',
              }]
            }
          ],
          ordered: true,
        }],
      },
    ],
  },
}

```

```
};  
});
```

新增 EXECUTE 意圖指令

在 `EXECUTE` 意圖中，新增 `action.devices.commands.SetModes` 指令，如以下程式碼片段所示。

index.js

```
const updateDevice = async (execution,deviceId) => {  
  ...  
  switch (command) {  
    ...  
    // Add SetModes command  
    case 'action.devices.commands.SetModes':  
      state = {load: params.updateModeSettings.load};  
      ref = getFirebaseRef().child(deviceId).child('Modes');  
      break;  
  }  
}
```

更新 QUERY 回應

接著，更新 `QUERY` 回應，回報洗衣機的目前狀態。

將更新後的變更新增至 `queryFirebase` 和 `queryDevice` 函式，以取得儲存在即時資料庫中的狀態。

index.js

```
const queryFirebase = async (deviceId) => {
  const snapshot = await getFirebaseRef().child(deviceId).once('value');
  const snapshotVal = snapshot.val();
  return {
    on: snapshotVal.OnOff.on,
    isPaused: snapshotVal.StartStop.isPaused,
    isRunning: snapshotVal.StartStop.isRunning,
    // Add Modes snapshot
    load: snapshotVal.Modes.load,
  };
};

const queryDevice = async (deviceId) => {
  const data = await queryFirebase(deviceId);
  return {
    on: data.on,
    isPaused: data.isPaused,
    isRunning: data.isRunning,
    currentRunCycle: [{ ... }],
    currentTotalRemainingTime: 1212,
    currentCycleRemainingTime: 301,
    // Add currentModeSettings
    currentModeSettings: {
      load: data.load,
    },
  };
};
```

更新報表狀態

最後，請更新 `reportstate` 函式，向住家圖表回報洗衣機目前的洗衣量設定。

[index.js](#)

```
const requestBody = {
  requestId: 'ff36a3cc', /* Any unique ID */
  agentUserId: USER_ID,
  payload: {
    devices: {
      states: {
        /* Report the current state of your washer */
        [context.params.deviceId]: {
          on: snapshot.OnOff.on,
          isPaused: snapshot.StartStop.isPaused,
          isRunning: snapshot.StartStop.isRunning,
          // Add currentModeSettings
          currentModeSettings: {
            load: snapshot.Modes.load,
          },
        },
      },
    },
  },
};
```

部署至 Firebase

執行下列指令，部署更新後的整合項目：

```
firebase deploy --only functions
```

部署完成後，前往網頁使用者介面，然後按一下工具列中的「重新整理」



按鈕。這會觸發「要求同步」，讓 Google 助理接收更新後的 SYNC 回應資料。

Smart Home Codelab 

現在你可以下達指令來設定洗衣機模式，例如：

「Ok Google，將洗衣機的洗衣量設為大。」

此外，你也可以詢問洗衣機相關問題，例如：

「Ok Google，洗衣機的衣物量是多少？」

注意： `currentRunCycle`、`currentTotalRemainingTime` 和 `currentCycleRemainingTime` 屬性與 `RunCycle` 特徵相關。在本程式碼研究室中，您不會對 `RunCycle` 特徵進行任何變更。在實際裝置上，這些值會根據裝置的目前狀態動態更新。

步驟 5 · 約 10 分鐘

5. 新增切換按鈕

`action.devices.traits.Toggles` 特徵代表裝置的具名方面，具有 `true` 或 `false` 狀態，例如洗衣機是否處於渦輪模式。

更新 SYNC 回應

在 `SYNC` 回應中，您需要新增有關新裝置特徵的資訊。如下列程式碼片段所示，這項資訊會顯示在 `traits` 陣列和 `attributes` 物件中。

[index.js](#)


```

app.onSync(body => {
  return {
    requestId: 'ff36a3cc-ec34-11e6-b1a0-64510650abcf',
    payload: {
      agentUserId: USER_ID,
      devices: [{
        id: 'washer',
        type: 'action.devices.types.WASHER',
        traits: [
          'action.devices.traits.OnOff',
          'action.devices.traits.StartStop',
          'action.devices.traits.RunCycle',
          'action.devices.traits.Modes',
          // Add Toggles trait
          'action.devices.traits.Toggles',
        ],
        name: { ... },
        deviceInfo: { ... },
        attributes: {
          pausable: true,
          availableModes: [{
            name: 'load',
            name_values: [{
              name_synonym: ['load'],
              lang: 'en'
            }],
            settings: [{ ... }],
            ordered: true,
          }],
          //Add availableToggles
          availableToggles: [{
            name: 'Turbo',
            name_values: [{
              name_synonym: ['turbo'],
              lang: 'en',
            }],
          }],
        },
      }],
    },
  };
});

```

新增 EXECUTE 意圖指令

在 EXECUTE 意圖中，新增 `action.devices.commands.SetToggles` 指令，如以下程式碼片段所示。

[index.js](#)

```

const updateDevice = async (execution,deviceId) => {
  ...
  switch (command) {
    \  ...
    // Add SetToggles command
    case 'action.devices.commands.SetToggles':
      state = {Turbo: params.updateToggleSettings.Turbo};
      ref = firebaseRef.child(deviceId).child('Toggles');
      break;
  }
}

```

更新 QUERY 回應

最後，您需要更新 `QUERY` 回應，回報洗衣機的渦輪模式。將更新後的變更新增至 `queryFirebase` 和 `queryDevice` 函式，以取得儲存在即時資料庫中的切換按鈕狀態。

index.js

```

const queryFirebase = async (deviceId) => {
  const snapshot = await getFirebaseRef().child(deviceId).once('value');
  const snapshotVal = snapshot.val();
  return {
    on: snapshotVal.OnOff.on,
    isPaused: snapshotVal.StartStop.isPaused,
    isRunning: snapshotVal.StartStop.isRunning,
    load: snapshotVal.Modes.load,
    // Add Toggles snapshot
    Turbo: snapshotVal.Toggles.Turbo,
  };
}

const queryDevice = async (deviceId) => {
  const data = queryFirebase(deviceId);
  return {
    on: data.on,
    isPaused: data.isPaused,
    isRunning: data.isRunning,
    currentRunCycle: [{ ... }],
    currentTotalRemainingTime: 1212,
    currentCycleRemainingTime: 301,
    currentModeSettings: {
      load: data.load,
    },
    // Add currentToggleSettings
    currentToggleSettings: {
      Turbo: data.Turbo,
    },
  };
};

```

更新報表狀態

最後，請更新 `reportstate` 函式，向 Home Graph 回報洗衣機是否設為快速模式。

index.js

```
const requestBody = {
  requestId: 'ff36a3cc', /* Any unique ID */
  agentUserId: USER_ID,
  payload: {
    devices: {
      states: {
        /* Report the current state of your washer */
        [context.params.deviceId]: {
          on: snapshot.OnOff.on,
          isPaused: snapshot.StartStop.isPaused,
          isRunning: snapshot.StartStop.isRunning,
          currentModeSettings: {
            load: snapshot.Modes.load,
          },
          // Add currentToggleSettings
          currentToggleSettings: {
            Turbo: snapshot.Toggles.Turbo,
          },
        },
      },
    },
  },
};
```

部署至 Firebase

執行下列指令來部署更新後的函式：

```
firebase deploy --only functions
```

部署完成後，按一下網頁版 UI 中的「重新整理」



按鈕，觸發「要求同步」作業。

你現在可以說出指令，將洗衣機設為快速模式：

「Ok Google，將洗衣機設為快速模式。」

你也可以詢問洗衣機是否已處於快速模式：

「Ok Google，洗衣機是否處於快速模式？」

6. 回報錯誤和例外狀況

透過雲端對雲端整合的錯誤處理功能，您可以在問題導致 `EXECUTE` 和 `QUERY` 回應失敗時，向使用者回報。使用者與智慧型裝置和整合服務互動時，通知可提供更優質的體驗。

只要 `EXECUTE` 或 `QUERY` 要求失敗，整合功能就應傳回錯誤代碼。舉例來說，如果使用者嘗試在洗衣機蓋子打開時啟動洗衣機，您想擲回錯誤，則 `EXECUTE` 回應會如以下程式碼片段所示：

```
{
  "requestId": "ff36a3cc-ec34-11e6-b1a0-64510650abcf",
  "payload": {
    "commands": [
      {
        "ids": [
          "456"
        ],
        "status": "ERROR",
        "errorCode": "deviceLidOpen"
      }
    ]
  }
}
```

現在，當使用者要求啟動洗衣機時，Google 助理會回應：

「洗衣機的蓋子沒蓋上。請關閉並重試。」

例外狀況與錯誤類似，但會指出警示是否與指令相關聯，且可能會或可能不會阻礙成功執行。例外狀況可以使用 `StatusReport` 特徵提供相關資訊，例如電池電量或最近的狀態變化。系統會傳回非阻斷例外狀況代碼和 `SUCCESS` 狀態，而阻斷例外狀況代碼則會與 `EXCEPTIONS` 狀態一併傳回。

以下程式碼片段顯示含有例外狀況的回應範例：

```
{
  "requestId": "ff36a3cc-ec34-11e6-b1a0-64510650abcf",
  "payload": {
    "commands": [{
      "ids": ["123"],
      "status": "SUCCESS",
      "states": {
        "online": true,
        "isPaused": false,
        "isRunning": false,
        "exceptionCode": "runCycleFinished"
      }
    }]
  }
}
```

Google 助理會說出以下內容：

「洗衣機已停止運轉。」

如要為洗衣機新增錯誤回報功能，請開啟 `functions/index.js`，並新增錯誤類別定義，如以下程式碼片段所示：

index.js

```
app.onQuery(async (body) => {...});

// Add SmartHome error handling
class SmartHomeError extends Error {
  constructor(errorCode, message) {
    super(message);
    this.name = this.constructor.name;
    this.errorCode = errorCode;
  }
}
```

更新執行回應，傳回錯誤代碼和錯誤狀態：

index.js

```
const executePromises = [];
const intent = body.inputs[0];
for (const command of intent.payload.commands) {
  for (const device of command.devices) {
    for (const execution of command.execution) {
      executePromises.push(
        updateDevice(execution, device.id)
          .then((data) => {
            ...
          })
      );
      //Add error response handling
      .catch((error) => {
        functions.logger.error('EXECUTE', device.id, error);
        result.ids.push(device.id);
        if (error instanceof SmartHomeError) {
          result.status = 'ERROR';
          result.errorCode = error.errorCode;
        }
      })
    );
  }
}
```

現在，Google 助理可以向使用者說明您回報的任何[錯誤代碼](#)。下一節會提供具體範例。

注意：雲端對雲端整合功能可能會發生裝置或全域層級的錯誤和例外狀況。本程式碼研究室只涵蓋單一洗衣機的實作方式，因此整合作業只會實作裝置層級的錯誤碼。詳情請參閱「[錯誤和例外狀況](#)」一節。

7. 新增雙重使用者驗證

如果裝置有任何需要保護的模式，或應僅限特定授權使用者群組使用，例如軟體更新或解除鎖定，您應在整合中實作[雙重使用者驗證](#)。

注意：雙重使用者驗證與 OAuth 不同，後者會將使用者的 Google 帳戶連結至驗證系統中的使用者帳戶。詳情請參閱「[使用 OAuth 連結帳戶](#)」。

您可以在所有裝置類型和特徵上導入雙重使用者驗證，並自訂安全驗證問題的觸發時機 (每次都觸發，或符合特定條件時觸發)。

系統[支援三種挑戰類型](#)：

- `No challenge`：不使用驗證挑戰的要求和回應 (這是預設行為)
- `ackNeeded` — 需要明確確認 (是或否) 的雙重使用者驗證
- `pinNeeded` — 需要個人識別碼 (PIN 碼) 的雙重使用者驗證

在本程式碼研究室中，請為開啟洗衣機的指令新增 `ackNeeded` 挑戰，並新增功能，在二次驗證挑戰失敗時傳回錯誤。

開啟 `functions/index.js`，並新增錯誤類別定義，傳回錯誤代碼和驗證類型，如以下程式碼片段所示：

`index.js`

```
class SmartHomeError extends Error { ... }

// Add secondary user verification error handling
class ChallengeNeededError extends SmartHomeError {
  /**
   * Create a new ChallengeNeededError
   * @param {string} suvType secondary user verification challenge type
   */
  constructor(suvType) {
    super('challengeNeeded', suvType);
    this.suvType = suvType;
  }
}
```

您也需要更新執行回應，傳回 `challengeNeeded` 錯誤，如下所示：

`index.js`

```

const executePromises = [];
const intent = body.inputs[0];
for (const command of intent.payload.commands) {
  for (const device of command.devices) {
    for (const execution of command.execution) {
      executePromises.push(

        updateDevice(execution, device.id)
          .then((data) => {
            ...
          })
          .catch((error) => {
            functions.logger.error('EXECUTE', device.id, error);
            result.ids.push(device.id);
            if (error instanceof SmartHomeError) {
              result.status = 'ERROR';
              result.errorCode = error.errorCode;
              //Add error response handling
              if (error instanceof ChallengeNeededError) {
                result.challengeNeeded = {
                  type: error.suvType
                };
              }
            }
          })
      );
    }
  }
}
}

```

最後，修改 `updateDevice`，要求明確確認是否要開啟或關閉洗衣機。

[index.js](#)


```
const updateDevice = async (execution,deviceId) => {
  const {challenge,params,command} = execution; //Add secondary user challenge
  let state, ref;
  switch (command) {
    case 'action.devices.commands.OnOff':
      //Add secondary user verification challenge
      if (!challenge || !challenge.ack) {
        throw new ChallengeNeededError('ackNeeded');
      }
      state = {on: params.on};
      ref = getFirestoreRef().child(deviceId).child('OnOff');
      break;
    ...
  }

  return ref.update(state)
    .then(() => state);
};
```

部署至 Firebase

執行下列指令來部署更新後的函式：

```
firebase deploy --only functions
```

部署更新後的程式碼後，要求 Google 助理開啟或關閉洗衣機時，必須口頭確認動作，例如：

你：「Ok Google，開啟洗衣機。」

Google 助理：「確定要啟動洗衣機嗎？」

你：「是。」

開啟 [Firebase 記錄](#)，即可查看雙重使用者驗證流程每個步驟的詳細回應。

2:25:12.441 PM		smarthome	▾ Unable to update washer. { ChallengeNeededError: ackNeeded at updateDevice (/srv/index.js:222:15) at app.onExecute (/srv/index.js:266:11) at Function.<anonymous> (/srv/node_modules/actions-on-google/dist/service/smarthome/smarthome.js:188:29) at Generator.next (<anonymous>) at /srv/node_modules/actions-on-google/dist/service/smarthome/smarthome.js:22:71 at new Promise (<anonymous>) at __awaiter (/srv/node_modules/actions-on-google/dist/service/smarthome/smarthome.js:18:12) at Function.handler (/srv/node_modules/actions-on-google/dist/service/smarthome/smarthome.js:174:16) at Object.<anonymous> (/srv/node_modules/actions-on-google/dist/assistant.js:55:32) at Generator.next (<anonymous>) name: 'ChallengeNeededError', errorCode: 'challengeNeeded', tfaType: 'ackNeeded' }
2:25:12.431 PM		smarthome	▸ Headers { "host": "us-central1-testshoodelab.cloudfunctions.net", "user-agent": "Mozilla/5.0 (compatible; Google-Cloud-Functi..." }
2:25:12.428 PM		smarthome	▸ Request { "inputs": [{ "context": { "locale_country": "US", "locale_language": "en" }, "intent": "action.devices.EXECUTE", "...

步驟 8 · 約 0 分鐘

8. 恭喜



恭喜！您透過 `Modes` 和 `Toggles` 特徵擴充了雲端對雲端整合功能，並透過雙重使用者驗證確保執行安全。

瞭解詳情

以下提供幾個可深入瞭解的構想：

- 在裝置中新增[本機執行](#)功能。
 - 使用其他雙重使用者驗證挑戰類型修改裝置狀態。
 - 更新 `RunCycle` 特徵 `QUERY` 回應，以動態更新。
 - 請參閱[這個 GitHub 範例](#)。
-